
Part 1 — Code Review & Debugging

What the endpoint is trying to do

- Create a new `Product`
- Create an initial `Inventory` row for that product in one warehouse
- Return success

Issues (technical + business logic), impact, and fixes

1) Wrong data model for multi-warehouse products

Problem: `Product` stores `warehouse_id`, implying a product belongs to only one warehouse. But requirement says: **Products can exist in multiple warehouses**.

Impact in production:

- You can't represent the same SKU/product stored in 2+ warehouses without duplicating products.
- Duplicated products break reporting (sales, procurement, reorder rules) and cause confusing UX ("Why do I have two Widget A's?").

Fix:

- Remove `warehouse_id` from `Product`.
- Use an association table like `inventory` (or `product_warehouse`) to represent quantities per warehouse.

---

2) No SKU uniqueness enforcement

Problem: Code just inserts `sku=data['sku']` with no validation and no DB constraint handling.

Impact:

- Two concurrent requests can create duplicate SKUs (race condition).
- Downstream integrations and picking/packing break because SKU is expected to uniquely identify an item.
- "Works in dev" but fails in production with real concurrency.

Fix:

- Enforce **DB unique constraint** on `products.sku` (platform-wide per requirement).
- Catch `IntegrityError` and return **409 Conflict**.

---

3) No input validation / missing fields cause 500s

Problem: Accesses `data['name']`, `data['sku']`, etc. If any are missing or `request.json` is `None`, it throws exceptions.

Impact:

- Random 500 errors when clients send partial payloads or wrong content-type.
- Operational noise and poor client experience.

Fix:

- Validate required fields, types, and ranges; return **400 Bad Request** with useful messages.

---

4) Price handling is unsafe for decimals

Problem: `price=data['price']` without converting/validating. Many systems accidentally use floats.

Impact:

- Floating precision bugs (e.g., 19.99 becomes 19.989999…).
- Incorrect invoices/taxes.

Fix:

- Use `Decimal` in app logic.
- Use DB type `NUMERIC(12,2)` (or similar).
- Validate non-negative price.

---

5) Two commits = broken atomicity

Problem: Commits product first, then inventory. If inventory insert fails, product is still created.

Impact:

- “Ghost products” that exist without inventory, leading to UI confusion and inconsistent state.
- Retries may create duplicates (especially if SKU uniqueness isn’t enforced).

Fix:

- Wrap both inserts in **one transaction**.
- Either commit once at the end or use `with db.session.begin():`.

---

6) No validation that warehouse exists / belongs to the correct company

Problem: Uses `warehouse_id` directly.

Impact:

- Orphan inventory records referencing non-existent warehouse (if FK not enforced) or DB errors.
- Cross-tenant data leakage if a user can create inventory in another company’s warehouse.

Fix:

- Check warehouse exists and is owned by the target company (requires auth context).
- Ensure FK constraints exist.

---

7) Initial quantity may be missing, negative, or non-integer

Problem: `quantity=data['initial_quantity']` without checks.

Impact:

- Negative on-hand counts at creation.
- Type errors at runtime.

Fix:

- Treat as optional (per prompt), default to 0.
- Validate integer and `>= 0`.

---

8) No handling for “product exists in multiple warehouses” at creation

Problem: API accepts a single `warehouse_id` + `initial_quantity`.

Impact:

- Real customers onboarding might want to initialize multiple warehouses at once.
- They'll spam the endpoint, or you'll duplicate products.

Fix:

- Support either:
  - a single `warehouse_id` + `initial_quantity` (backward compatible), OR
  - an `inventories: [{warehouse_id, quantity}, ...]` array.

---

9) Response/status code not ideal

Problem: Returns a dict (Flask will JSONify in newer versions, but better explicit), doesn't set HTTP 201.

Impact:

- Clients can't reliably interpret creation semantics.
- Poor API hygiene.

Fix:

- Return JSON + `201 Created`.
- Optionally include created resource representation.

Corrected Version (Flask + SQLAlchemy)

This version assumes:

- `Product` is **not tied to a warehouse**
- `Inventory` has a **unique constraint** on `(product_id, warehouse_id)`
- `Product.sku` is **unique globally**
- Optional support for `inventories` array

```
from decimal import Decimal, InvalidOperation
from flask import request, jsonify
from sqlalchemy.exc import IntegrityError
from sqlalchemy import select

@app.route("/api/products", methods=["POST"])
```

```
def create_product():
    data = request.get_json(silent=True)
    if not data:
        return jsonify(error="Invalid or missing JSON body"), 400

    # --- Basic field validation ---
    required = ["name", "sku"]
    missing = [k for k in required if not data.get(k)]
    if missing:
        return jsonify(error=f"Missing required fields: {',
'.join(missing)}"), 400

    name = str(data["name"]).strip()
    sku = str(data["sku"]).strip().upper()

    # Price is optional per prompt; default to 0.00
    raw_price = data.get("price", "0.00")
    try:
        price = Decimal(str(raw_price))
    except (InvalidOperation, TypeError):
        return jsonify(error="price must be a decimal value"), 400
    if price < 0:
        return jsonify(error="price must be >= 0"), 400

    # Backward-compatible single-warehouse payload:
    # { warehouse_id, initial_quantity }
    # New multi-warehouse payload:
    # { inventories: [{warehouse_id, quantity}, ...] }
    inventories_payload = data.get("inventories")
    if inventories_payload is None:
        # If they didn't pass inventories[], we accept legacy fields
(optional).
        wh_id = data.get("warehouse_id")
        init_qty = data.get("initial_quantity", 0)

        if wh_id is None:
            # warehouse is optional; allow product creation without inventory
            inventories = []
        else:
            try:
                init_qty_int = int(init_qty)
            except (TypeError, ValueError):
                return jsonify(error="initial_quantity must be an integer"),
400
            if init_qty_int < 0:
                return jsonify(error="initial_quantity must be >= 0"), 400
            inventories = [{"warehouse_id": wh_id, "quantity": init_qty_int}]
    else:
        if not isinstance(inventories_payload, list):
            return jsonify(error="inventories must be an array"), 400
        inventories = []
        for i, row in enumerate(inventories_payload):
            if not isinstance(row, dict):
                return jsonify(error=f"inventories[{i}] must be an object"),
400
            if "warehouse_id" not in row:
```

```
                return jsonify(error=f"inventories[{i}].warehouse_id is
required"), 400
            try:
                qty = int(row.get("quantity", 0))
            except (TypeError, ValueError):
                return jsonify(error=f"inventories[{i}].quantity must be an
integer"), 400
            if qty < 0:
                return jsonify(error=f"inventories[{i}].quantity must be >=
0"), 400
            inventories.append({"warehouse_id": row["warehouse_id"],
"quantity": qty})

    try:
        # --- Atomic transaction: product + inventory ---
        with db.session.begin():
            # Create product (no warehouse_id here)
            product = Product(name=name, sku=sku, price=price)

            db.session.add(product)
            db.session.flush()  # ensures product.id is available without
committing

            # Validate warehouses exist (and belong to company if multi-
tenant auth exists)
            # Example check (pseudo):
            # allowed_warehouse_ids = set(...)
            # if any(w not in allowed_warehouse_ids for w in ...): return 403

            for inv in inventories:
                # Ensure warehouse exists
                warehouse = db.session.execute(
                    select(Warehouse).where(Warehouse.id ==
inv["warehouse_id"])
                ).scalar_one_or_none()
                if not warehouse:
                    # Raising will rollback transaction
                    raise ValueError(f"Warehouse {inv['warehouse_id']} not
found")

                inventory = Inventory(
                    product_id=product.id,
                    warehouse_id=inv["warehouse_id"],
                    quantity=inv["quantity"],
                )
                db.session.add(inventory)

        return jsonify(
            message="Product created",
            product_id=product.id,
        ), 201

    except ValueError as e:
        db.session.rollback()
        return jsonify(error=str(e)), 400

    except IntegrityError:
```

```
        db.session.rollback()
        # Likely SKU unique violation or inventory unique violation
        return jsonify(error="SKU already exists (must be unique)"), 409
```

Key changes explained:

- **Transaction** wraps everything: either both product + inventory succeed or nothing is saved.
- Uses `Decimal` for price.
- SKU normalized and uniqueness handled via DB constraint.
- Supports product creation **with no inventory** (optional fields).
- Supports **multi-warehouse initialization** via `inventories[]`.

Part 2 — Database Design

Goals from requirements

- Multi-tenant: companies → warehouses
- Products stored in multiple warehouses with quantities
- Track inventory level changes over time (audit trail)
- Suppliers provide products
- Bundles made of other products

Proposed Schema (PostgreSQL-flavored SQL DDL)

1) Companies + Warehouses

```
CREATE TABLE companies (
  id            BIGSERIAL PRIMARY KEY,
  name          TEXT NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE warehouses (
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name          TEXT NOT NULL,
  address       TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (company_id, name)
);

CREATE INDEX idx_warehouses_company ON warehouses(company_id);
```

2) Products + product types (threshold varies by type)

```
CREATE TABLE product_types (
```

```
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT REFERENCES companies(id) ON DELETE CASCADE, -- allow
per-company types; or NULL for global catalog
  name          TEXT NOT NULL
);

CREATE TABLE products (
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name          TEXT NOT NULL,
  sku           TEXT NOT NULL,
  product_type_id BIGINT REFERENCES product_types(id),
  price         NUMERIC(12,2) NOT NULL DEFAULT 0,
  is_bundle     BOOLEAN NOT NULL DEFAULT FALSE,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- Requirement says SKU unique across platform:
  UNIQUE (sku)
  -- If instead SKU is only unique per company, use: UNIQUE(company_id, sku)
);

CREATE INDEX idx_products_company ON products(company_id);
CREATE INDEX idx_products_type ON products(product_type_id);
```

3) Inventory (current snapshot) + movements (history/audit)

```
CREATE TABLE inventory (
  id            BIGSERIAL PRIMARY KEY,
  product_id    BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  warehouse_id  BIGINT NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
  quantity      INTEGER NOT NULL DEFAULT 0 CHECK (quantity >= 0),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (product_id, warehouse_id)
);

CREATE INDEX idx_inventory_wh ON inventory(warehouse_id);
CREATE INDEX idx_inventory_product ON inventory(product_id);

-- Immutable ledger of changes (best for audit + debugging)
CREATE TABLE inventory_movements (
  id            BIGSERIAL PRIMARY KEY,
  product_id    BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  warehouse_id  BIGINT NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
  delta         INTEGER NOT NULL, -- positive or negative
  reason        TEXT NOT NULL,    -- e.g. "sale", "purchase_receipt",
"adjustment", "transfer_in", "transfer_out"
  reference_type TEXT,            -- e.g. "sales_order", "purchase_order"
  reference_id   BIGINT,
  actor_user_id  BIGINT,          -- optional (if you have users)
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_movements_prod_wh_time ON inventory_movements(product_id,
warehouse_id, created_at DESC);
```

Why both `inventory` and `inventory_movements`?

- `inventory` = fast reads for "current stock"
- `inventory_movements` = audit/history + supports "track when inventory levels change"
- Keep them consistent via application logic (transaction updates both) or DB triggers.

4) Suppliers + mapping to products

```
CREATE TABLE suppliers (
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name          TEXT NOT NULL,
  contact_email TEXT,
  phone         TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE(company_id, name)
);

CREATE INDEX idx_suppliers_company ON suppliers(company_id);

-- Many-to-many: a supplier can supply many products, and a product can have
many suppliers
CREATE TABLE product_suppliers (
  product_id    BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  supplier_id   BIGINT NOT NULL REFERENCES suppliers(id) ON DELETE CASCADE,
  is_primary    BOOLEAN NOT NULL DEFAULT FALSE,
  supplier_sku  TEXT,
  lead_time_days INTEGER,
  PRIMARY KEY (product_id, supplier_id)
);

CREATE INDEX idx_product_suppliers_primary ON product_suppliers(product_id)
WHERE is_primary = TRUE;
```

5) Bundles (product contains other products)

```
CREATE TABLE bundle_components (
  bundle_product_id     BIGINT NOT NULL REFERENCES products(id) ON DELETE
CASCADE,
  component_product_id  BIGINT NOT NULL REFERENCES products(id) ON DELETE
RESTRICT,
  component_qty         NUMERIC(12,3) NOT NULL CHECK (component_qty > 0),
  PRIMARY KEY (bundle_product_id, component_product_id)
);

-- Prevent self-reference in app logic (bundle cannot contain itself)
```

6) Sales activity (needed for low-stock alerts rule)

You need *some* way to compute "recent sales activity" and "days until stockout". A minimal model:

```
CREATE TABLE sales_orders (
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
```

```
  warehouse_id  BIGINT NOT NULL REFERENCES warehouses(id),
  status        TEXT NOT NULL, -- "completed", "cancelled", etc.
  completed_at  TIMESTAMPTZ,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_sales_orders_company_time ON sales_orders(company_id,
completed_at DESC);

CREATE TABLE sales_order_items (
  id            BIGSERIAL PRIMARY KEY,
  sales_order_id BIGINT NOT NULL REFERENCES sales_orders(id) ON DELETE
CASCADE,
  product_id    BIGINT NOT NULL REFERENCES products(id),
  quantity      NUMERIC(12,3) NOT NULL CHECK (quantity > 0)
);

CREATE INDEX idx_sales_items_product ON sales_order_items(product_id);
```

7) Reorder thresholds (varies by product type, and often overridden per product/warehouse)

```
CREATE TABLE reorder_policies (
  id            BIGSERIAL PRIMARY KEY,
  company_id    BIGINT NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  product_type_id BIGINT REFERENCES product_types(id),
  product_id    BIGINT REFERENCES products(id),
  warehouse_id  BIGINT REFERENCES warehouses(id),
  threshold_qty INTEGER NOT NULL CHECK (threshold_qty >= 0),
  lookback_days INTEGER NOT NULL DEFAULT 30 CHECK (lookback_days > 0),
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),

  -- exactly one scope should be set in practice; enforce via app or partial
constraints
  -- precedence idea: warehouse+product > product > product_type
);

CREATE INDEX idx_reorder_policy_lookup
ON reorder_policies(company_id, product_id, warehouse_id, product_type_id);
```

Gaps / questions to ask the product team

These are the “requirements intentionally incomplete” items I’d clarify early:

Tenancy + SKU

1. Is `sku` truly **global unique across platform**, or **unique per company**? (Global uniqueness is unusual for B2B multi-tenant.)
2. Can two different companies sell the same SKU like “COKE-20OZ”? If yes, uniqueness must be `(company_id, sku)`.

Inventory semantics
3) Can inventory go negative? (Backorders, oversells, adjustments.) If yes, remove `CHECK(quantity >= 0)` and treat negative as allowed.
4) Do we need units of measure (each, case, kg, liters)? (Especially important for bundles and suppliers.)

Bundles
5) Are bundles "virtual" (only explode on sale) or do they have their own inventory count?
6) Are bundle components fixed or configurable?

Suppliers
7) Can a product have multiple suppliers and preferred ordering rules (primary vs backup)?
8) Do we track supplier pricing tiers, MOQ, lead time, order cutoffs?

Tracking changes
9) Which events create `inventory_movements`? (sales completion, purchase receipt, transfer, adjustment)
10) Do we need to support transfers between warehouses as one logical event (transfer_out + transfer_in linked by reference)?

Low stock alerts
11) Definition of "recent sales activity": last 7/14/30 days? completed orders only? include refunds?
12) How to compute "days_until_stockout": based on average daily sales, median, or forecast model?
13) Threshold source: product type default, per product override, per warehouse override—what precedence?

Security
14) Roles/permissions: who can create products and modify inventory? Are warehouses restricted per user?

Design decisions & scalability notes

- **Inventory snapshot + ledger** is a proven pattern: fast reads plus auditability.
- **Composite unique (product_id, warehouse_id)** prevents duplicates and ensures clean upserts.
- **Indexes on time + product/warehouse** power alert queries efficiently.
- **Reorder policy precedence** allows business flexibility without schema rewrites.
- Prefer DB constraints for correctness under concurrency (SKU uniqueness especially).

Part 3 — Low-Stock Alerts API

Endpoint

```
GET /api/companies/{company_id}/alerts/low-stock
```

Assumptions (documented)

1. **Current stock** is stored in `inventory.quantity`.
2. "Recent sales activity" means: product has sales in the last `lookback_days` (default 30) and we compute average daily usage from completed orders.
3. Threshold is resolved in this precedence order:
   - warehouse+product policy
   - product policy
   - product_type policy
   - fallback: 0 (no alert) or a company default (I'll choose "no alert" if no threshold found).
4. Supplier: pick `product_suppliers.is_primary = true` if present; else any supplier; else supplier is `null`.
5. Multi-warehouse: alert is per **(product, warehouse)** row.
6. We only alert if `avg_daily_sales > 0` (otherwise days_until_stockout is undefined/infinite).

Implementation (Flask + SQLAlchemy)

This is a readable implementation; in production you'd likely refine into fewer queries/CTEs, but this is correct and explainable in an interview.

```
from flask import jsonify
from sqlalchemy import func, case, and_, select
from datetime import datetime, timedelta, timezone

@app.route("/api/companies/<int:company_id>/alerts/low-stock",
methods=["GET"])
def low_stock_alerts(company_id: int):
    """
    Returns low-stock alerts per warehouse for a company.

    Business rules:
    - Low stock threshold varies by product type (and may be overridden by
product / warehouse)
    - Only alert for products with recent sales activity
    - Handle multiple warehouses per company
    - Include supplier info for reordering

    Assumptions:
    - sales_orders.status='completed' and sales_orders.completed_at is set
    - sales_order_items.quantity is the sold quantity
    - inventory.quantity is current on-hand
    """
```

```
    # In a real app: enforce authz (caller must have access to this
company_id)

    now = datetime.now(timezone.utc)

    # --- 1) Pull current inventory rows for this company (joined to product
+ warehouse) ---
    inv_rows = db.session.execute(
        select(
            Inventory.product_id,
            Inventory.warehouse_id,
            Inventory.quantity.label("current_stock"),
            Product.name.label("product_name"),
            Product.sku,
            Product.product_type_id,
            Warehouse.name.label("warehouse_name"),
        )
        .join(Product, Product.id == Inventory.product_id)
        .join(Warehouse, Warehouse.id == Inventory.warehouse_id)
        .where(
            Product.company_id == company_id,
            Warehouse.company_id == company_id,
        )
    ).all()

    if not inv_rows:
        return jsonify(alerts=[], total_alerts=0), 200

    # Collect ids for later queries
    product_ids = list({r.product_id for r in inv_rows})
    warehouse_ids = list({r.warehouse_id for r in inv_rows})

    # --- 2) Resolve reorder thresholds (policy precedence) ---
    # We fetch policies and resolve in Python for clarity.
    policies = db.session.execute(
        select(
            ReorderPolicy.product_id,
            ReorderPolicy.warehouse_id,
            ReorderPolicy.product_type_id,
            ReorderPolicy.threshold_qty,
            ReorderPolicy.lookback_days,
        )
        .where(ReorderPolicy.company_id == company_id)
    ).all()

    # Build lookup maps
    wh_prod_policy = {}   # (product_id, warehouse_id) -> (threshold,
lookback)
    prod_policy = {}      # product_id -> (threshold, lookback)
    type_policy = {}      # product_type_id -> (threshold, lookback)

    for p in policies:
        key_wp = (p.product_id, p.warehouse_id)
        if p.product_id and p.warehouse_id:
            wh_prod_policy[key_wp] = (p.threshold_qty, p.lookback_days)
        elif p.product_id:
```

```
            prod_policy[p.product_id] = (p.threshold_qty, p.lookback_days)
        elif p.product_type_id:
            type_policy[p.product_type_id] = (p.threshold_qty,
p.lookback_days)

    def resolve_policy(product_id, warehouse_id, product_type_id):
        # precedence: warehouse+product > product > type > None
        if (product_id, warehouse_id) in wh_prod_policy:
            return wh_prod_policy[(product_id, warehouse_id)]
        if product_id in prod_policy:
            return prod_policy[product_id]
        if product_type_id in type_policy:
            return type_policy[product_type_id]
        return None  # no threshold configured

    # --- 3) Compute sales velocity for "recent sales activity" and
days_until_stockout ---
    # We compute sold_qty over last N days. Because N may vary by policy,
    # we pick a conservative max lookback across policies (default 30).
    default_lookback = 30
    max_lookback = max([p.lookback_days for p in policies],
default=default_lookback)
    start_time = now - timedelta(days=max_lookback)

    sales = db.session.execute(
        select(
            SalesOrder.warehouse_id,
            SalesOrderItem.product_id,
            func.sum(SalesOrderItem.quantity).label("qty_sold"),
            func.max(SalesOrder.completed_at).label("last_sold_at"),
        )
        .join(SalesOrderItem, SalesOrderItem.sales_order_id == SalesOrder.id)
        .where(
            SalesOrder.company_id == company_id,
            SalesOrder.status == "completed",
            SalesOrder.completed_at >= start_time,
            SalesOrder.warehouse_id.in_(warehouse_ids),
            SalesOrderItem.product_id.in_(product_ids),
        )
        .group_by(SalesOrder.warehouse_id, SalesOrderItem.product_id)
    ).all()

    # Map: (warehouse_id, product_id) -> {qty_sold, last_sold_at}
    sales_map = {(s.warehouse_id, s.product_id): s for s in sales}

    # --- 4) Supplier info (primary supplier if available) ---
    # We fetch supplier mapping for the involved products.
    suppliers = db.session.execute(
        select(
            ProductSupplier.product_id,
            Supplier.id.label("supplier_id"),
            Supplier.name.label("supplier_name"),
            Supplier.contact_email.label("supplier_email"),
            ProductSupplier.is_primary,
        )
        .join(Supplier, Supplier.id == ProductSupplier.supplier_id)
        .join(Product, Product.id == ProductSupplier.product_id)
```

```
        .where(
            Product.company_id == company_id,
            ProductSupplier.product_id.in_(product_ids),
        )
        .order_by(ProductSupplier.product_id,
ProductSupplier.is_primary.desc())
    ).all()

    # Choose the first row per product due to ordering (primary first)
    supplier_map = {}
    for row in suppliers:
        if row.product_id not in supplier_map:
            supplier_map[row.product_id] = {
                "id": row.supplier_id,
                "name": row.supplier_name,
                "contact_email": row.supplier_email,
            }

    # --- 5) Build alerts ---
    alerts = []

    for r in inv_rows:
        policy = resolve_policy(r.product_id, r.warehouse_id,
r.product_type_id)
        if not policy:
            # No threshold configured => do not alert
            continue

        threshold, lookback_days = policy
        # Must have recent sales activity: we interpret as "sold at least
once within lookback_days"
        sale = sales_map.get((r.warehouse_id, r.product_id))
        if not sale:
            continue

        lookback_start = now - timedelta(days=lookback_days)
        if not sale.last_sold_at or sale.last_sold_at < lookback_start:
            continue

        # Compute avg daily sales using the policy lookback period
        # Note: sale.qty_sold is over max_lookback window; we approximate by
using the same qty
        # if lookback varies. For strict correctness, run per-lookback
aggregation or store daily rollups.
        qty_sold = float(sale.qty_sold or 0)
        avg_daily_sales = qty_sold / float(lookback_days) if lookback_days >
0 else 0.0

        if avg_daily_sales <= 0:
            continue

        current_stock = int(r.current_stock)

        # Only alert if current_stock < threshold (or <= depending on
business)
        if current_stock >= threshold:
            continue
```

```
        days_until_stockout = int(current_stock / avg_daily_sales) if
avg_daily_sales > 0 else None

        alerts.append({
            "product_id": r.product_id,
            "product_name": r.product_name,
            "sku": r.sku,
            "warehouse_id": r.warehouse_id,
            "warehouse_name": r.warehouse_name,
            "current_stock": current_stock,
            "threshold": int(threshold),
            "days_until_stockout": days_until_stockout,
            "supplier": supplier_map.get(r.product_id)  # may be None
        })

    return jsonify(alerts=alerts, total_alerts=len(alerts)), 200
```

Edge cases to consider (and how I'd handle them)

1. **No policies configured**

- Current implementation returns no alerts.
- Alternative: use a company default threshold.

2. **No recent sales data**

- We skip alerts per rule "only alert for products with recent sales activity".
- If the business later wants "alert anyway if below threshold", that becomes a toggle.

3. **Lookback varies per product**

- The code approximates by using a single aggregated window.
- For correctness at scale:
    - store **daily sales rollups** (product_id, warehouse_id, day, qty_sold)
    - compute exact lookback sums cheaply.

4. **Bundles**

- Big question: do bundles affect component stock at sale time?
    - If yes: sales velocity for components should include "bundle explosions".
    - You'd implement this in order completion logic (inventory_movements).

5. **Stockouts / zero stock**

- days_until_stockout becomes 0 (if current_stock is 0).
- You might want to cap at 0 and still alert.

6. **Supplier missing**

- Response includes `"supplier": null`.
- Or you can omit the field; spec shows supplier object, but null is safer than lying.

7. **Multi-tenant authorization**

- Must ensure the caller can read `company_id`.
- Must ensure warehouses/products joined belong to that company (I included `where` checks).

8. **Performance**

- For large datasets:
  - convert Python loops into one SQL query with CTEs + joins
  - add indexes:
    - `sales_orders(company_id, status, completed_at)`
    - `sales_order_items(product_id)`
    - `inventory(product_id, warehouse_id)`
    - `reorder_policies(company_id, product_id, warehouse_id, product_type_id)`

Summary of Assumptions (as requested)

- Product is company-scoped; SKU uniqueness is enforced **globally** only because the prompt explicitly says so (I'd confirm; it's commonly per-company).
- Inventory is tracked per product+warehouse; changes are recorded in `inventory_movements`.
- Recent sales activity is based on completed orders within a configurable lookback.
- Threshold resolution uses a clear precedence: warehouse+product > product > product type.